PayCore

Ledger Hardening and Payment Orchestration
Implementation report, step by step

Date	5 October 2026
Repository	`paycore`, branch `main` (changes uncommitted)
Roadmap phases covered	Phase 1 (ledger hardening), Phase 2 (system account validation), Phase 3 (payment domain), Phase 4 (external payment reliability)
Migrations added	`V25__harden_ledger.sql`, `V26__create_payments.sql`
Test suite	777 tests, 0 failures, 1 skipped (baseline: 743, 0 failures)

Contents

1. Summary
2. Reading the roadmap against the code
3. Step 0: baseline
4. Phase 1: ledger hardening
5. Phase 2: validating the system accounts
6. Phase 3: the payment domain
7. Phase 4: external payment reliability
8. API, configuration and UI changes
9. Verification
10. Defects fixed along the way
11. Limitations and what comes next
12. Appendix: files changed

PayCore is a learning and portfolio system. The card processor and bank rail used here are simulators; no real money moves.

1. Summary

The roadmap proposed moving PayCore "from a ledger to a payment processing platform". Before changing anything I read the code to find where it actually stood. Most of the roadmap's early phases were already built. The real gaps were narrower, and more serious, than the roadmap suggested:

- The ledger's invariants were enforced **only by application code**. Nothing in the database stopped an unbalanced transaction, a posted transaction with no entries, or an edit to historical entries.
- There was **no payment lifecycle**. When the bank rail timed out on a transfer, the service logged one line, told the customer "PENDING", and then **nothing ever came back to it**. The customer's money stayed debited indefinitely.
- When the card provider timed out, PayCore answered `502` with "*you have not been charged*". That claim was **not knowable**: a timeout can happen after the charge succeeds.
- Two concurrent top-up requests with the same idempotency key **both charged the card**. Only one was credited, and the other became a log line asking someone to refund it.

The work done, in order:

Phase	What was built
1. Ledger hardening	Migration V25 makes PostgreSQL enforce four ledger invariants: balanced transactions, no posted transaction without entries, append-only entries, and forward-only transaction status. Eight new integration tests cover concurrency, idempotency under load, duplicate webhooks, racing reversals, immutability and a mixed-workload trial balance.
2. System accounts	Validated rather than rebuilt. A new test proves that after a concurrent mixed workload, the settlement account equals the sum of what PayCore owes its customers.
3. Payment domain	New `payment` package with a `Payment` entity, a six-state machine (`CREATED, PROCESSING, UNKNOWN, NEEDS_REVIEW, SUCCEEDED, FAILED`), enforced in Java and in the database. A database-written audit trail records every status change, and existing provider payments are backfilled.
4. Reliability	Status-query contracts on both providers. Outbound transfers and card top-ups were rewritten as orchestrations over the state machine. A recovery engine settles unknown outcomes by asking the provider, with exponential backoff, and handles payments interrupted by a crash. Payments that can't be settled automatically go to operations. Staff endpoints were added to inspect and re-check payments.

2
Flyway migrations (V25, V26)

25
new integration tests (ledger + payment + migration)

777
tests in the full suite

0
failures

> **Deliberately not done**
> Phase 5 (reconciliation), Phase 6 (transactional outbox) and Phase 7 (Kafka) were not started. The roadmap orders them after the payment domain, and each needs the payment table built here as its foundation. Section 11 explains what each would need.

2. Reading the roadmap against the code

The roadmap's own table marks most phases as complete. I checked each claim against the source before deciding what was "needed".

Roadmap item	Status found	Evidence in the code
Double-entry ledger	Built	`LedgerService` posts a debit and an equal credit in one transaction. Balances are derived from entries, and amounts are BIGINT minor units (V18).
Concurrency / no double spend	Built	`SELECT … FOR UPDATE` on accounts in id order (deadlock-free), with funds re-checked under the lock. Two concurrency tests existed.
Idempotency	Built	Unique `(initiated_by, idempotency_key)`, with replay comparing every field. Provider payments are unique by `(provider, provider_reference)`.
Ownership	Built	The source account is only found if it belongs to the caller. Tests exist.
Currency safety	Built	Composite foreign keys `(account_id, currency)` and `(transaction_id, currency)`.
Frozen/closed accounts	Built	Parameterised tests on source and destination state.
Reversal keeps originals	App only	The code never edits entries, but the **database allowed** UPDATE and DELETE on them.
Balanced transactions	App only	`LedgerTransaction.post()` checks the balance, but the **database accepted** unbalanced entries.
System / settlement accounts	Built	V19: settlement accounts per currency, created race-free on first use.
Funding provider, bank rail	Built	`FundingProvider` and `BankRail` interfaces with simulators.
Maker-checker	Built	`operations_requests` with a four-eyes CHECK (V24).
Payment domain / state machine	Missing	No payment entity. A transfer's outcome lived only in the HTTP response.
Timeout / unknown outcome	Missing	`BankTransferService`: "*needs a status check*" was logged, but no status check existed anywhere.
Crash recovery	Missing	A crash between the debit and the rail call left the money debited, with no record of whether it was sent.
Reconciliation, outbox, Kafka	Later	Not started, as the roadmap intends.

> **Conclusion**
> "Finish ledger hardening" meant moving the invariants into the database and proving them under concurrency. "Payment orchestration" was genuinely missing, and its absence caused customer-visible failures. Those became the scope.

3. Step 0: baseline

Before any change, I started Docker Desktop (the integration tests use Testcontainers with PostgreSQL 18) and ran the full suite to have a reference point.

Run	Tests	Failures	Errors	Skipped
Before any change	743	0	0	1 (Dojah sandbox, needs real credentials)

4. Phase 1: ledger hardening

1 Migration V25: the database enforces the ledger's invariants

The application already followed these rules. V25 makes PostgreSQL refuse to break them, so a bug, a stray SQL statement or a future code path cannot corrupt the books. This is the same role the unique constraints already played for idempotency.

Guarantee 1: every transaction balances (deferred constraint trigger)

A deferred constraint trigger on `ledger_entries` runs at **COMMIT**. It runs then because the service inserts the transaction, then the debit, then the credit, and only the complete set must balance. It sums debits and credits for the transaction and raises `check_violation` if they differ or are zero.

```
CREATE CONSTRAINT TRIGGER trg_ledger_entries_balanced
    AFTER INSERT ON ledger_entries
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION paycore_ledger_entries_balanced();
```

Guarantee 2: a POSTED transaction has entries

A second deferred trigger, on `ledger_transactions`, applies the same balance check to every transaction inserted with a status other than `INITIATED`. This catches the failure where the header is saved but the entries never are.

Guarantee 3: entries are append-only

A `BEFORE UPDATE OR DELETE` trigger on `ledger_entries` refuses both operations. A mistake is corrected by posting a compensating transaction.

Guarantee 4: a transaction's facts are immutable, and its status only moves forward

On `ledger_transactions`, the trigger refuses DELETE and refuses changes to 16 "fact" columns: reference, type, currency, initiator, key, description, staff note, approver, provider, provider reference, counterparty, creation time and reversal link. It allows only the status transitions `INITIATED → POSTED` and `POSTED → REVERSED`. Once `posted_at` or `reversed_at` is set, it can't change.

> **Two honest caveats, documented in the migration**
> (1) A maintenance escape hatch exists: `SET LOCAL paycore.ledger_maintenance = 'on'`, which lasts only for one database transaction and is never set by the application. Test fixtures use it to back-date statement entries.
> (2) Row triggers don't fire on `TRUNCATE`, and a table owner can bypass them. In production, the application's database role should not own these tables and should lack DELETE and TRUNCATE. No trigger can enforce that.

2 Adapting test fixtures to the stricter database

Five integration test classes reset the database with `DELETE FROM ledger_entries`, which V25 now (correctly) refuses. They were changed to `TRUNCATE ledger_entries, ledger_transactions CASCADE`, the table owner's reset, which also clears the tables that reference them. In `LedgerIntegrationTest`:

- `backdate()` now runs inside a transaction with the maintenance flag set.
- A uniqueness test that inserted a POSTED transaction with no entries now inserts it with balanced entries in one transaction (`insertBalancedTransaction`). The empty case became its own negative test.

3 New tests: proving each invariant

All were added to the existing `LedgerIntegrationTest` rather than to a parallel class.

Invariant (roadmap)	Test	What it proves
Idempotency: same request, one movement	`concurrentRetriesOfOneRequestShouldMoveMoneyOnce`	Ten identical transfers with one key, fired simultaneously, return the same transaction id, with one ledger row and the balances moved once.
Concurrency: no double spending	`manyConcurrentTransfersShouldNeverOverdrawTheAccount`	40 parallel ₦3,000 transfers from ₦100,000: exactly 33 succeed and 7 fail with insufficient funds. ₦1,000 remains, and recipients received exactly ₦99,000.
Duplicate webhook: one movement	`aNotificationDeliveredManyTimesAtOnceShouldBeCreditedOnce`	The same rail session delivered 10 times concurrently creates one transaction and one credit.
Reversal is single	`twoApprovedReversalsOfOneTransactionShouldReverseItOnce`	Two separately approved reversal requests race: one posts, the other is refused as already reversed.

Double-entry under load	`booksShouldBalanceAfterAMixedConcurrentWorkload`	30 concurrent mixed operations (inbound, internal, outbound) leave no unbalanced transaction, a ledger that nets to zero, and no negative balance.
Reversal keeps originals	`postedEntriesShouldNeverChangeEvenWhenReversed`	A byte-level snapshot of the original entries is identical after reversal. The reversal has its own two entries.
Immutable history (DB)	`databaseShouldRefuseToRewriteLedgerHistory`	Direct SQL UPDATE or DELETE on entries is refused, as are edits to transaction facts, deletes, and status moving backwards.
Balanced (DB)	`databaseShouldRefuseAnUnbalancedTransaction`	Debit 100 with credit 99 is refused at commit, a POSTED transaction with no entries is refused, and nothing is persisted.

The roadmap's ownership, currency and account-state invariants already had passing tests (`customerCannotTransferFromSomeoneElsesAccount`, `transferCurrencyMustMatchTheAccounts`, `sourceAccountMustBeActive` and `destinationAccountMustBeActive`), so they were verified rather than duplicated.

5. Phase 2: validating the system accounts

The roadmap noted that system accounts were already built structurally and needed validation, not construction. The key property of a settlement account is a **trial balance**: the money PayCore holds at its bank must equal the money it owes its customers.

`booksShouldBalanceAfterAMixedConcurrentWorkload` asserts exactly this after a concurrent burst. It checks that the settlement balance read through the staff API equals the sum of all customer balances. The existing tests `settlementShouldAlwaysEqualWhatPayCoreOwesItsCustomers` (bank transfers) and the settlement checks in the funding and operations suites continue to pass.

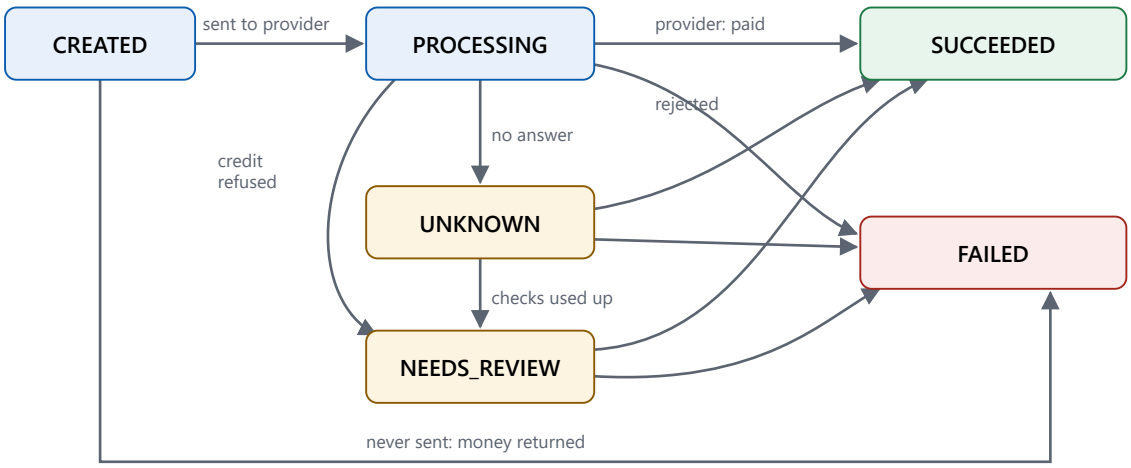
As the roadmap advised, no further system accounts (fee revenue, suspense, chargeback, clearing) were added. Settlement is sufficient for this stage.

6. Phase 3: the payment domain

> **Payment vs ledger**
> The **ledger** records what money moved: accounting entries. A **payment** records the business process around a call to an external provider: what PayCore asked for, what the provider answered, and whether the answer is still outstanding. A payment exists for every transfer out to another bank and every card top-up. Transfers between two PayCore accounts never leave PayCore, so they stay ledger-only.

```
payment PAY-20261005-8F3A2C91D0B47E15   card top-up of N5,000      status PROCESSING
ledger  TXN-20261005-1C0E77A2B9D4F031   DEBIT settlement N5,000    CREDIT customer N5,000
```

4 The state machine



Allowed transitions. SUCCEEDED and FAILED are final. A top-up starts in PROCESSING, because nothing is reserved before the provider call. A transfer starts in CREATED, because the customer is debited first.

State	Meaning
`CREATED`	Recorded. For a transfer, the customer is already debited, but nothing has been sent to the rail.
`PROCESSING`	Committed *before* the provider is called. From this point PayCore must assume the provider may have acted.
`UNKNOWN`	The provider didn't answer (timeout or outage), or PayCore stopped before recording the answer. **Never treated as failed.** A status check is always scheduled (`next_check_at` is required).
`NEEDS_REVIEW`	Automatic checks gave up, or the customer was charged but couldn't be credited. Operations must look.
`SUCCEEDED`	Final. Always linked to its ledger posting.
`FAILED`	Final. Always has a customer-safe reason. A failed transfer always has the reversal that returned the money.

The rules are defined once in `PaymentStatus.allowedNext()` and checked by every mutating method on the `Payment` entity (`startProcessing` , `succeed` , `fail` , `markUnknown` , `requireReview`). An illegal move throws `InvalidPaymentTransitionException` , which maps to HTTP 409 `PAYMENT_STATE_CONFLICT` .

5 Migration V26: the payments table, enforced in the database

The `payments` table holds what was asked for (type, customer, account, amount, currency, key, provider, beneficiary), the provider identifiers, the ledger links, and recovery bookkeeping (`status_checks`, `next_check_at`). An optimistic `version` column completes it. The key constraints:

Constraint	Purpose
`uk_payments_customer_idempotency_key`	One key, one payment, however many retries or concurrent duplicates arrive. This is what stops a second card charge.
`uk_payments_provider_external_reference`	One provider payment (NIP session id or merchant reference) is one PayCore payment.
`fk_payments_account (account_id, currency)`	A payment's currency is its account's currency.
`ck_payments_completed_at`	`completed_at` is set exactly when the status is final.
`ck_payments_unknown_scheduled`	An UNKNOWN payment always has its next check scheduled.
`ck_payments_succeeded_posted`	Success always has its ledger posting.
`ck_payments_outbound_shape`	A transfer names its beneficiary, is debited from the start, and if FAILED has its reversal.
`ck_payments_funding_shape`	A top-up has no beneficiary or reversal, and if SUCCEEDED has the provider's reference.

State machine trigger (`trg_payments_guard`): the transitions from step 4, enforced again by PostgreSQL. It refuses DELETE, refuses changes to what was asked for, and refuses any change once the payment is final.

Audit trail trigger (`trg_payments_audit`): every insert and every status change writes a `payment_events` row (from, to, check count, reason, time). Because the database writes it, no code path can move a payment without leaving a record. The application only reads these rows, through the read-only `PaymentEvent` entity.

Backfill. Every existing `OUTBOUND_TRANSFER` and provider `DEPOSIT` in the ledger gets a settled payment, so the table is complete from day one, which is the base reconciliation will need. A reversed transfer becomes FAILED, linked to its reversal. For top-ups, the SQL recomputes the merchant reference exactly as the Java code does: `'PC-'` plus the first 16 bytes of `SHA-256("<customer>:<key>")` in hex. `PaymentBackfillMigrationIntegrationTest` proves the two match.

7. Phase 4: external payment reliability

6 Status-query contracts on both providers

An UNKNOWN outcome can only be settled by asking the provider. Each provider interface gained the operation real providers offer:

Interface	New method	Answers
`BankRail` (NIP)	`RailStatus status(String sessionId)` Transaction status query (TSQ)	`SUCCESSFUL`, `FAILED(reason)`, `PENDING`, `NOT_FOUND`
`FundingProvider`	`FundingStatus status(String merchantReference)`	`CONFIRMED(ref)`, `DECLINED(reason)`, `PENDING`, `NOT_FOUND`

Both simulators now remember what they did, keyed by session id or merchant reference, and deduplicate repeat requests as real providers do. The kobo of the amount select the scenario, so every path can be tried from the UI or API:

Kobo	Bank rail (transfer out)	Card provider (top-up)
`.99`	Rejected by the beneficiary bank (unchanged)	Declined (unchanged)
`.98` new	Paid, then the answer is lost (timeout)	Charged, then the answer is lost (timeout)
`.97` new	Times out before reaching the rail; nobody is paid	Times out before reaching the provider; nothing is charged
other	Paid	Confirmed

7 Outbound transfer, rewritten as an orchestration

`BankTransferService.send` (the existing class, updated) now drives the state machine. No database transaction or row lock is held while the rail answers.

```
1. payment for this key exists?  same request: return its outcome, pay nothing
                                 different request: 409 IDEMPOTENCY_KEY_REUSED
2. name enquiry                  the beneficiary must exist
3. ONE db transaction            ledger: DEBIT customer, CREDIT settlement (account locked)
                                 payment: CREATED, linked to the debit
4. commit PROCESSING             before the rail is called
5. rail.send(...)                no db transaction open
6. paid                          SUCCEEDED
   rejected                      ONE db transaction: reversal + FAILED
   exception / timeout           UNKNOWN, first status check scheduled -> customer sees PENDING
```

Steps 3 and 6 (rejected) are single database transactions. A crash can leave a payment *unfinished*, but never money moved without its payment, and never a payment marked failed without its money returned.

8 Card top-up, rewritten as an orchestration

```
1. ledger replay / validation    already credited? return it; else own, active, currency, limit
2. payment for this key exists?  return its outcome (pending, credited, or declined again); no
charge
3. insert payment PROCESSING     unique key: a concurrent duplicate fails here, not at the card
4. provider.collect(...)         no db transaction open
5. confirmed                     ONE db transaction: ledger credit + SUCCEEDED      -> 201 deposit
   declined                      FAILED                                              -> 422
PAYMENT_DECLINED
   exception / timeout           UNKNOWN                                             -> 202
Accepted + payment
6. credit refused after charge   NEEDS_REVIEW "charged but not credited"; operations credit or
refund
```

The HTTP contract only gains a case: `201` with the deposit is unchanged, and `202 Accepted` is new, with the payment body and a `Location: /api/v1/payments/{id}` header the customer can follow.

9 The recovery engine

`PaymentRecoveryService.recoverDue()` runs every 30 seconds through `PaymentRecoveryScheduler`. Each sweep:

Finds	Does
`CREATED` older than `stale-after`	The request stopped *before* the rail call, which only happens after PROCESSING is committed, so it was never sent. The debit is reversed and the payment FAILED: "*could not be sent; your money was returned*".
`PROCESSING` older than `stale-after`	The request stopped mid-call, so the provider may have acted. The payment moves to UNKNOWN with a check due now.
`UNKNOWN` with `next_check_at` due	It claims the check (counts it and schedules the next with exponential backoff), asks the provider, and applies the answer.

Answer	Transfer	Top-up
Paid / confirmed	SUCCEEDED (debit stands)	Credit + SUCCEEDED (once)
Rejected / declined	Reversal + FAILED	FAILED
Not found	Believed only after `not-found-grace` (10 min), because a provider may not have recorded a request it just received. After that: never arrived, so the transfer is reversed + FAILED and the top-up is FAILED ("not charged").	
Pending / no answer	Stays UNKNOWN. After `max-checks` (12) the payment goes to NEEDS_REVIEW, and a transfer stays debited, because the beneficiary may have been paid.	

Safe on several instances at once

- Every state change locks the payment row (`SELECT … FOR UPDATE`) and re-checks the state under the lock. For example, "abandon an unsent transfer" can't overtake a request thread that is just now sending it.
- Claiming a check pushes `next_check_at` into the future in its own commit, so a second instance skips it.
- Transitions that races make legitimately repeatable (the request thread and a status check both learning that a payment succeeded) are idempotent.
- Lock order is always payment row, then ledger rows, and no path takes them in reverse, so there are no deadlock cycles.

Operations

Staff can list payments by status (for example the `NEEDS_REVIEW` queue), read a payment's full event history, and ask the provider again (`POST /admin/payments/{id}/status-check`, permission `LEDGER_REQUEST`). Staff never *decide* an outcome. The provider's answer moves the payment and any money, in keeping with PayCore's "treat it as a real bank" principle.

The roadmap's reliability scenarios, and how each is now handled

Scenario	Behaviour now	Proven by

Provider timeout after acting	UNKNOWN. A status check finds it paid: the transfer is SUCCEEDED, or the top-up is credited once. Never reversed.	`aTimeoutAfterTheRailPaid…`, `aTopUpChargedButUnanswered…`
Provider timeout before receiving	UNKNOWN. A status check finds nothing: the transfer is reversed, and the top-up fails with "not charged".	`aTimeoutBeforeTheRail…`, `aTopUpThatNeverReached…`
PayCore crashes before sending	Stale CREATED: reversed, FAILED.	`aTransferDebitedButNeverSent…`
Provider says SUCCESS, PayCore crashes	Stale PROCESSING, then UNKNOWN, then a status check, then SUCCEEDED or credited.	`aTransferLeftProcessing…`, `aTopUpChargedBeforePayCoreStopped…`
Duplicate webhook (×10)	One credit.	`aNotificationDeliveredManyTimes…`
Duplicate concurrent top-ups	One payment, one charge, one credit.	`concurrentTopUpsWithOneKey…`
Provider status never answers	NEEDS_REVIEW after max-checks. An operations re-check applies the provider's answer later.	`checksThatNeverGetAnAnswer…`
Charged but account frozen meanwhile	NEEDS_REVIEW "charged but not credited".	`aChargeThatCannotBeCredited…`

8. API, configuration and UI changes

New endpoints

Endpoint	Permission	Purpose
`GET /api/v1/payments/{id}`	`TRANSACTION_READ`	Customer's own payment (404 for anyone else's)
`GET /api/v1/payments/reference/{ref}`	`TRANSACTION_READ`	The same, by PAY- reference
`GET /api/v1/admin/payments?status=&page=&size=`	`ACCOUNT_VIEW_ALL`	Payment list, newest first, filterable by status
`GET /api/v1/admin/payments/{id}`, `/reference/{ref}`	`ACCOUNT_VIEW_ALL`	Any payment
`GET /api/v1/admin/payments/{id}/events`	`ACCOUNT_VIEW_ALL`	Status history, oldest first
`POST /api/v1/admin/payments/{id}/status-check`	`LEDGER_REQUEST`	Ask the provider now; 409 while still in its request

Changed responses (backwards compatible)

- **Bank transfer** response: new fields `paymentId`, `paymentReference` and `paymentStatus`. `transferStatus` keeps its three values (SUCCESSFUL, FAILED, PENDING).
- **Top-up**: the new `202 Accepted` case described in step 8.
- New errors: `404 PAYMENT_NOT_FOUND`, `409 PAYMENT_STATE_CONFLICT`.
- The provider-outage message no longer claims "you have not been charged".

Configuration (`application.properties`)

```
paycore.payments.recovery.enabled=${PAYCORE_PAYMENT_RECOVERY_ENABLED:true}
paycore.payments.recovery.interval=${PAYCORE_PAYMENT_RECOVERY_INTERVAL:PT30S}
paycore.payments.stale-after=2m        # must exceed the longest provider call
paycore.payments.first-check-after=30s
paycore.payments.max-backoff=30m       # 30s, 1m, 2m, 4m ... capped
paycore.payments.max-checks=12         # then NEEDS_REVIEW
paycore.payments.not-found-grace=10m
```

In `pom.xml`, Surefire sets `paycore.payments.recovery.enabled=false`, so the timer never runs during tests. Tests run sweeps explicitly and stay deterministic.

UI (`paycore-ui`)

- `types.ts`: new `Payment` and `PaymentStatus` types, and the payment fields on `OutboundTransfer`.
- `AddMoney.tsx`: a `202` top-up shows "Confirming your payment" with its PAY- reference instead of being mis-read as a deposit. The hint text mentions the new `.98` and `.97` test amounts. The transfer page already rendered PENDING, so it needed no change.
- Type-checks cleanly (`tsc -b`).

9. Verification

Test class	Tests	Status	Kind

`LedgerIntegrationTest`	41 (8 new)	Pass	PostgreSQL 18 via Testcontainers
`PaymentIntegrationTest` new	16	Pass	PostgreSQL + simulated rail and card provider
`PaymentBackfillMigrationIntegrationTest` new	1	Pass	Flyway V25 → V26 on real data
`FundingServiceTest` (rewritten)	14	Pass	Unit, mocked collaborators
`SimulatedFundingProviderTest`	8 (4 new)	Pass	Unit
Full suite	**777**	**Pass (0 failures, 1 skipped)**	Before: 743, 0 failures

> **One failure found and fixed during verification**
> The first full run reported 777 tests with 1 failure: `FundingIntegrationTest.concurrentTopUpsShouldNeverPassTheLimit`. The cause was the change of `FundingService.fund` to return `FundingOutcome` instead of `TransactionResponse`. The test collected results as `Object`, so the compiler couldn't flag it, and its type filter matched nothing. The money behaviour was correct (3 credited, 3 refused, balance ₦900). The test now unwraps the outcome and also asserts the new guarantee: a top-up refused after its card was charged ends in `NEEDS_REVIEW`, never lost.

Counts are taken from the Surefire reports of the final run.

10. Defects fixed along the way

Defect	Impact before	Now
Rail timeout logged, then forgotten	The customer was debited, the response said PENDING, and nothing ever settled it. The money stayed out until someone read the logs.	UNKNOWN, with a scheduled status check, then reversed or confirmed.
"You have not been charged" on a card provider timeout	A false statement to the customer whenever the charge had in fact succeeded. The paid money was never credited.	202 pending. A status check credits a confirmed charge.
Concurrent duplicate top-ups both charged	Two card charges for one key. One was credited, and the other became a "refund it" log line.	The unique payment key stops the second request before the card is charged.
"Paid but not credited" existed only in logs	No queue for operations to work from.	NEEDS_REVIEW payments, listable through the admin API.
Ledger invariants enforced only in Java	Any other writer could unbalance or rewrite history.	PostgreSQL enforces them (V25).

11. Limitations and what comes next

Known limitations

- **NEEDS_REVIEW has no resolution workflow yet.** Operations can see a payment and ask the provider again. If the provider confirms a charge PayCore can't credit, the fix today is an adjustment through the existing maker-checker flow, and the payment itself stays in review. A dedicated "resolve with refund/credit" request under maker-checker is the natural next step.
- **Simulators remember in memory.** After a restart they've forgotten earlier sessions, so a transfer interrupted mid-call before a restart would be judged "not found" and reversed. A real provider keeps this record. This affects the demo only.
- **The backfill maps any reversed outbound transfer to FAILED,** including one reversed later by operations rather than rejected by the rail. The money outcome is the same, and the payment's narrative is simplified.
- **The database triggers protect against bugs, not against a table owner.** Production must also restrict privileges, as section 4 explains.
- **Webhook-driven confirmation** for card providers (as opposed to polling) isn't built. The roadmap's "duplicate webhook" case is covered for the bank rail's inbound webhook.

Recommended order for the remaining roadmap

Phase	What it now has to build on
5. Reconciliation	Every provider payment, including history thanks to the backfill, has a `payments` row keyed by `(provider, external_reference)`, with amount and final status. A reconciliation engine can match a provider settlement file against it and classify each line as `MATCHED`, `MISSING_LOCAL`, `MISSING_PROVIDER`, `AMOUNT_MISMATCH` or `STATUS_MISMATCH`. It needs a settlement-report method on each provider interface and a `reconciliation_items` table.
6. Transactional outbox	The single-transaction boundaries are already identified (debit + payment, reversal + FAILED, credit + SUCCEEDED). An `outbox_events` insert belongs in exactly those transactions, and `payment_events` shows the shape of the events.
7. Kafka	Only once a consumer exists that needs it (notifications, fraud), published from the outbox, as the roadmap says.

> **Not committed**
> All changes are in the working tree on `main` and haven't been committed, so they can be reviewed first.

12. Appendix: files changed

New: database

`db/migration/V25__harden_ledger.sql`, `db/migration/V26__create_payments.sql`

New: payment package (`com.fintechplatform.paycore.payment`)

`enums/PaymentStatus`, `enums/PaymentType`	State machine and payment types
`entity/Payment`, `entity/PaymentEvent`	Aggregate with guarded transitions; read-only audit row
`repository/PaymentRepository`, `PaymentEventRepository`	Lookups, row lock, stale and due-check queries

`service/PaymentService`	Every state change, each in one short locked transaction
`service/PaymentRecoveryService`, `PaymentRecoveryScheduler`	Recovery sweep and manual check; 30-second timer
`controller/PaymentController`, `AdminPaymentController`	Customer and staff endpoints
`dto/PaymentResponse`, `PaymentEventResponse`, `PaymentPageResponse`	API shapes
`config/PaymentProperties`, `PaymentConfiguration`	Recovery timing and backoff
`exception/PaymentNotFoundException`, `InvalidPaymentTransitionException`	404 and 409

New: elsewhere

`banktransfer/rail/RailStatus`, `funding/provider/FundingStatus`, `funding/service/FundingOutcome`

Modified

`BankTransferService`, `BankRail`, `SimulatedBankRail`, `OutboundTransferResponse`, `FundingService`, `FundingController`, `FundingProvider`, `SimulatedFundingProvider`, `LedgerService` (dead `findOutbound` removed), `GlobalExceptionHandler`, `application.properties`, `pom.xml`, `README.md`, `paycore-ui/src/api/types.ts`, `paycore-ui/src/pages/app/AddMoney.tsx`

Tests

New: `PaymentIntegrationTest`, `PaymentBackfillMigrationIntegrationTest`. Extended: `LedgerIntegrationTest`, `SimulatedFundingProviderTest`, `FundingIntegrationTest`. Rewritten: `FundingServiceTest`. Cleanup adapted to V25: `BankTransferIntegrationTest`, `FundingIntegrationTest`, `OperationsIntegrationTest`, `DemoModeIntegrationTest`.